Software & Qualitätsmanagement

Dokumentation RISC-V Emulator

Gruppe: C

Mitglieder: Julius, Amgad, Emanuel, Justin, Alex

Datum: 30.04.2026

Inhaltsverzeichnis

1. Gruppenmeetings 3
2. Dokumentation zur Pipeline und zu den Test-Cases 6
2.1 Projektüberblick 6
3. CI-Pipeline 6
3.1 Zweck der Pipeline 6
3.2 Build-Stage 7
3.3 Static-Test mit Cppcheck 7
3.4 Dynamic-Test mit CUnit 8
3.5 Deploy-Stage 9
4. Implementierte Test-Suites 10
4.1 Init-Test-Suite 10
4.2 Decode-Test-Suite 11
4.3 Execute-Test-Suite 11
4.4 Jump-Test-Suite 12
5. Fazit 12
6. Quellenverzeichnis: 13

1. Gruppenmeetings

Meeting 1 – Projektstart und Aufgabenverteilung

Ziel des Meetings:
Im ersten Meeting wurde das bestehende RISC-V-Emulator-Projekt ausgewählt und die Aufgabenstellung gemeinsam besprochen. Außerdem wurden die Rollen innerhalb der Gruppe verteilt.

Besprochene Punkte:

- Auswahl des bestehenden RISC-V-Emulator-Projekts
- Überblick über den vorhandenen Code
- Besprechung der Anforderungen aus der Aufgabenstellung
- Grobe Planung der GitLab-CI-Pipeline
- Aufgabenverteilung innerhalb der Gruppe

Aufgabenverteilung:

Person	Aufgabe
Julius	Technische Umsetzung, Build, Tests, Fehlerbehebung
Amgad	Organisation, Koordination, Ansprechpartner
Alex	Dokumentation und organisatorische Unterstützung
Emanuel	Präsentation und Recherche
Justin	Recherche und theoretische Unterstützung

Ergebnis:
Die Gruppe hatte einen gemeinsamen Überblick über das Projekt und die Anforderungen. Die Rollen wurden verteilt und die nächsten technischen Schritte wurden festgelegt.

Meeting 2 – Umsetzung der Pipeline und Tests

Ziel des Meetings:
Im zweiten Meeting wurde der Fortschritt bei der CI-Pipeline und den automatisierten Tests besprochen.

Besprochene Punkte:

- Build-Stage in GitLab CI
- Einbindung von Cppcheck für statische Codeanalyse
- Ausgabe von Cppcheck als XML-Datei
- Umwandlung der XML-Datei in eine JSON-Datei
- Einbindung von CUnit für dynamische Tests
- Auswahl sinnvoller Testfälle
- Speicherung von Build- und Testergebnissen als Artefakte

Ergebnis:
Die wichtigsten Pipeline-Schritte wurden umgesetzt oder vorbereitet. Cppcheck und CUnit wurden als zentrale Werkzeuge für statische und dynamische Tests festgelegt.

Meeting 3 – Abschluss, Kontrolle und Dokumentation

Ziel des Meetings:
Im dritten Meeting wurden die Ergebnisse kontrolliert, die Dokumentation fertiggestellt und die Präsentation geplant.

Besprochene Punkte:

- Prüfung der Pipeline-Stages
- Kontrolle der Testergebnisse
- Behebung kleinerer Fehler
- Besprechung der Artefakte in GitLab
- Fertigstellung der Dokumentation
- Vorbereitung der Präsentation
- Reflexion über die Arbeitsweise

Ergebnis:
Die Pipeline wurde final geprüft, die Dokumentation wurde überarbeitet und abgeschlossen.

2. Dokumentation zur Pipeline und zu den Test-Cases

2.1 Projektüberblick

Für das Projekt wurde ein bestehender RISC-V Emulator in C verwendet. Das Programm war bereits vorhanden und grundsätzlich funktionsfähig. Im Rahmen der Gruppenarbeit wurde das Projekt in GitLab eingebunden und um eine automatisierte CI-Pipeline erweitert.

Das Ziel war es, das bestehende Projekt automatisch bauen, statisch analysieren, testen und ausführen zu können.

Im Verlauf des Projekts wurde wiederholt die offizielle RISC-V-Dokumentation von RISC-V International herangezogen. Sie diente als Grundlage, um die verwendeten RV32-Instruktionen besser zu verstehen und die Testfälle fachlich einzuordnen [1].

Die Pipeline besteht ausfolgenden Stages:

Static-Test (linting) → Dynamic-Test → Build

3. CI-Pipeline

3.1 Zweck der Pipeline

Die GitLab-CI-Pipeline dient dazu, wichtige Schritte der Softwareentwicklung automatisch auszuführen. Dadurch wird sichergestellt, dass das Projekt nach Änderungen weiterhin gebaut werden kann und die Tests erfolgreich laufen.

Die Pipeline übernimmt folgende Aufgaben:

- Kompilieren des Projekts
- Statische Codeanalyse mit Cppcheck
- Ausführen automatisierter Tests mit CUnit
- Speichern von Ergebnissen als Artefakte

3.2 Build-Stage

In der Build-Stage wird der RISC-V Emulator kompiliert. Dafür wird das vorhandene Makefile verwendet.

Ziel der Build-Stage:

- Prüfen, ob der Source Code fehlerfrei kompiliert
- Sicherstellen, dass alle benötigten Dateien vorhanden sind
- Build-Ergebnis als Artefakt speichern

Typischer Befehl:

make

Benötigte Pakete:

gcc
make
libncurses-dev

Ergebnis:
Wenn die Build-Stage erfolgreich ist, wurde der Emulator erfolgreich erstellt.

3.3 Static-Test mit Cppcheck

Für die statische Codeanalyse wurde Cppcheck verwendet. Cppcheck analysiert den Quellcode, ohne das Programm auszuführen.

Ziel von Cppcheck:

- Finden möglicher Programmierfehler
- Erkennen unsauberer oder gefährlicher Codestellen
- Verbesserung der Codequalität
- Prüfung von Source Code und Testcode

Die Ausgabe von Cppcheck wird als XML-Datei erzeugt:

cppcheck . --xml 2> cppcheck_result.xml

cppcheck emittiert die gefundenen Fehler via stderr, deshalb muss ‘2>’ verwendet werden, um den gewünschten Output in eine Report-File umzulenken.

Da GitLab für bestimmte Auswertungen bzw. Artefakte eine passende Datei benötigt, wird die XML-Datei anschließend in eine JSON-Datei umgewandelt. Dafür wurde ein eigenes Python-Skript verwendet.

Ablauf:

Cppcheck läuft über den Code
→ Ergebnis wird als XML gespeichert
→ XML wird mit Python eingelesen
→ JSON-Datei wird erzeugt
→ XML und JSON werden als Artefakte gespeichert

Ergebnis:
Die statische Analyse läuft automatisch in der Pipeline. Die Ergebnisse werden abgelegt und können nach jedem Pipeline-Lauf überprüft werden. Dadurch ist nachvollziehbar, ob durch Änderungen neue Warnungen oder Probleme im Code entstanden sind.

3.4 Dynamic-Test mit CUnit

Für die dynamischen Tests wurde CUnit verwendet. Im Gegensatz zu Cppcheck wird der Code hier tatsächlich ausgeführt. Dadurch kann geprüft werden, ob zentrale Funktionen des RISC-V Emulators korrekt arbeiten.

Die Tests befinden sich im Ordner:

cunit/

Die zentrale Datei für die Registrierung der Tests ist:

riscv_test_registry.c

Dort werden die einzelnen Test-Suites eingebunden und die Test-Cases registriert.

Ziel der Dynamic-Tests:

- Automatisiertes Testen einzelner Funktionen
- Überprüfung wichtiger Emulatorfunktionen
- Sicherstellen, dass Änderungen keine bestehenden Funktionen beschädigen

Typischer Ablauf:

```
cd cunit
make
./riscv_test.bin
```

Ergebnis:
Die CUnit-Tests werden automatisch ausgeführt. Die Testergebnisse können als Artefakte gespeichert werden.

3.5 Deploy-Stage

Wurde bewusst nicht implementiert da der Emulator bislang nur mit User-Input funktioniert. Diese Einschränkung stammt aus dem ursprünglichen Verwendungszweck des Programms. Um einen automatisierten Test durchführen zu können müssten entweder Key-Inputs simuliert werden, oder der Code stark verändert werden – was aus Zeitgründen während der Projektplanung abgelehnt wurde.

4. Implementierte Test-Suites

Die CUnit-Tests wurden in mehrere Test-Suites aufgeteilt:

Test-Suite	Zweck
Init-Test-Suite	Prüft Initialisierung, Register-Reset und Speicher-Reset
Decode-Test-Suite	Prüft das Dekodieren von RISC-V-Instruktionen
Execute-Test-Suite	Prüft die Ausführung ausgewählter Befehle
Jump-Test-Suite	Prüft Programmzähler und Sprungbefehle

4.1 Init-Test-Suite

Die Init-Test-Suite prüft, ob der Emulator korrekt vorbereitet wird.

Testcase	Beschreibung
init_state_test	Prüft, ob der Emulator nach der Initialisierung sinnvolle Startwerte besitzt.
init_regs_test	Prüft, ob alle Register nach dem Reset den Wert 0 enthalten.
init_memory_test	Prüft, ob der Speicher korrekt auf 0 gesetzt wird.

Diese Tests sind wichtig, weil alle weiteren Emulator Funktionen auf einem korrekt initialisierten Zustand aufbauen.

4.2 Decode-Test-Suite

Die Decode-Test-Suite prüft, ob RISC-V-Befehle korrekt in ihre Bestandteile zerlegt werden.

Testcase	Beschreibung
decode_nop_test	Prüft das Dekodieren eines NOP-Befehls bzw. ADDI x0, x0, 0.
decode_imm_negative_test	Prüft einen negativen Immediate-Wert.
decode_imm_positive_test	Prüft einen positiven Immediate-Wert.
decode_imm_posmax_test	Prüft den maximalen positiven Immediate-Wert.
decode_imm_negmin_test	Prüft den minimalen negativen Immediate-Wert.
decode_imm_random_test	Prüft mehrere zufällige Immediate-Werte.

Diese Tests sind wichtig, da der Emulator Instruktionen zuerst korrekt dekodieren muss, bevor sie ausgeführt werden können

4.3 Execute-Test-Suite

Die Execute-Test-Suite prüft, ob ausgewählte Instruktionen korrekt ausgeführt werden.

Testcase	Beschreibung
execute_addi_rand_test	Prüft die Ausführung von ADDI mit zufälligen Immediate-Werten.
execute_add_rand_test	Prüft die Ausführung von ADD mit zufälligen Registerwerten.

Diese Tests prüfen grundlegende arithmetische Funktionen des Emulators. Besonders ADDI und ADD eignen sich dafür, da sie zentrale Basisbefehle im RISC-V-Befehlssatz sind.

4.4 Jump-Test-Suite

Die Jump-Test-Suite prüft das Verhalten des Programmzählers und eines Sprungbefehls.

Testcase	Beschreibung
jumps_inc_pc_test	Prüft, ob der Programm Counter korrekt um 4 erhöht wird.
jumps_rand_jal_test	Prüft die Ausführung eines JAL-Sprungs mit zufälligen Sprungwerten.

Diese Tests sind wichtig, weil Sprungbefehle den Programmfluss verändern. Der Emulator muss daher den Programm Counter korrekt berechnen und setzen.

5. Fazit

Im Projekt wurden sowohl Static Tests als auch Dynamic Tests umgesetzt. Für die Static Tests wurde Cppcheck verwendet, um den C-Code automatisch auf mögliche Fehler und problematische Codestellen zu prüfen. Die Ergebnisse werden als XML-Datei erzeugt und anschließend in JSON umgewandelt, damit sie in der Pipeline als Artefakt abgelegt und weiterverwendet werden können.

Für die Dynamic Tests wurde CUnit eingesetzt. Die Tests wurden in mehrere Test-Suites aufgeteilt, welche die Initialisierung, das Dekodieren von Befehlen, die Ausführung von Instruktionen und Sprungbefehle prüfen. Dadurch werden zentrale Funktionen des RISC-V Emulators automatisiert getestet.

Die Pipeline führt Static-Test, Dynamic-Test und Build automatisch aus. Dadurch ist nach jedem Pipeline-Lauf nachvollziehbar, ob das Projekt gebaut werden kann, ob die Codeanalyse erfolgreich ist und ob die implementierten Tests bestehen.

6. Quellenverzeichnis:

[1] RISC-V International: RV32/64G Instruction Set Listings, https://docs.riscv.org/reference/isa/unpriv/rv32.html, abgerufen am 30.04.2026.